

RootFin – Project Documentation

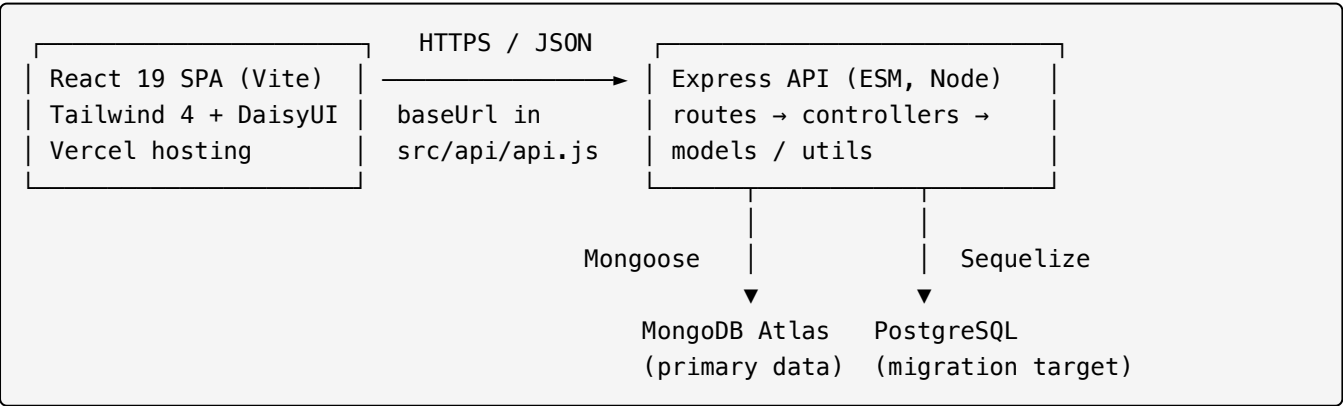
RootFin is a retail ERP / finance platform built by **Rootments** for a chain of stores (shoe / suit / wedding-wear retail). It covers daily cash & bank accounting (Day Book, closures, security deposits), purchasing, sales, inventory (items, groups, adjustments, transfers, store orders) and reporting.

This repository (`RootFin`) contains the project: a React SPA (`frontend/`) and a Node/Express API (`backend/`) that can talk to MongoDB, PostgreSQL, or both.

Table of Contents

- 1. [Architecture](#)
- 2. [Repository Layout](#)
- 3. [Tech Stack & Dependencies](#)
- 4. [Getting Started](#)
- 5. [Configuration](#)
- 6. [Backend in Detail](#)
- 7. [Frontend in Detail](#)
- 8. [Domain Modules](#)
- 9. [Roles, Access Control & Guards](#)
- 10. [CI/CD & Deployment](#)
- 11. [Maintenance Scripts](#)
- 12. [Housekeeping Files & Known Issues](#)
- 13. [Git Workflow](#)

1. Architecture



- **Frontend** is a client-routed SPA; session user is kept client-side (`currentuser`) and each route is guarded by role/store checks.
- **Backend** is a layered Express app. A `DB_TYPE` switch (`mongodb` | `postgresql` | `both`) selects which databases are connected at startup.

- **Mongoose models** (`backend/model/`) are the primary data layer; **Sequelize models** (`backend/models/sequelize/`) mirror a subset for the PostgreSQL migration (see §6.4).
- In-memory caching (`node-cache`) is used for heavy report endpoints.

2. Repository Layout

```

RootFin/
├─ README.md                ← this document
├─ package.json             root-level deps (cookie-parser, cors, express, mongoose,
nodemailer, react-*, vite)
├─ .github/workflows/
│   └─ deploy.yml           rsync + PM2 deploy to EC2 on push to master
│   └─ daily-commit-report.yml daily Conventional-Commits audit
├─ .vscode/settings.json    editor settings
├─ backend/
│   └─ server.js            app entry point
│   └─ swagger.js          Swagger UI at /api-docs
│   └─ .env                environment variables (not committed to docs)
│   └─ db/                 database.js (Mongo), postgresql.js (Sequelize)
│   └─ route/              Express routers (24 files)
│   └─ controllers/        request handlers (≈30 files)
│   └─ model/              Mongoose schemas (≈25 files)
│   └─ models/sequelize/   Sequelize models (11 files + index.js)
│   └─ utils/              stock engines, numbering, cache, e-mail
│   └─ utlis/              legacy helpers (parseBase64, nextInvoice) – note typo in
folder name
│   └─ scripts/            one-off DB maintenance / migration scripts
├─ frontend/
│   └─ index.html, vite.config.js, vercel.json, eslint.config.js
│   └─ public/             static assets
│   └─ src/
│       └─ main.jsx, App.jsx bootstrap + all routes
│       └─ api/api.js       API base URL
│       └─ components/      shared UI (Nav, Header, guards, uploads...)
│       └─ pages/           ≈100 page components
│       └─ hooks/           data-fetching & UX hooks
│       └─ utils/           cache, alerts, warehouse mapping, image upload
│       └─ config/          salesInventoryAccess.json (feature gating)
│       └─ data/            openingBalance.json
├─ production_changes.patch patch captured when syncing production → test
└─ fix_home.cjs / fix_report.cjs / scratch.jsx  ad-hoc dev helpers

```

3. Tech Stack & Dependencies

Backend (backend/package.json , ES modules)

Package	Purpose
express 4	HTTP server
mongoose 8	MongoDB ODM
sequelize 6 + pg	PostgreSQL ORM / driver
bcrypt	password hashing
cookie-parser, cors, body-parser	HTTP middleware
dotenv, cross-env	env loading / cross-platform <code>NODE_ENV</code>
node-cache	in-memory report cache
nodemailer	reorder-alert & PO e-mails
node-fetch	outbound HTTP
swagger-jsdoc, swagger-ui-express	API docs
nodemon	dev auto-reload

Scripts: `npm start` (production, 2 GB heap), `npm run dev` (nodemon, development), `check-transfers`, `optimize:quick`, `optimize:full` (index creation), `test:stock`.

Frontend (frontend/package.json , Vite)

Package	Purpose
react 19, react-dom, react-router-dom 7	UI + routing
tailwindcss 4 (@tailwindcss/vite), daisyui 4	styling
react-select	searchable dropdowns
react-toastify	notifications
recharts	charts
lucide-react, react-icons	icons
jspdf, html2pdf.js, html2canvas-pro	PDF export (html2canvas-pro adds <code>oklch()</code> colour support)
react-to-print, react-csv	print / CSV export
html5-qrcode	barcode / QR scanning
react-helmet	document head

Scripts: `dev`, `build`, `lint`, `preview`.

Root package.json

Holds a duplicate set of shared libs (cookie-parser, cors, express 5, mongoose, nodemailer, react-csv, react-helmet, react-icons, vite). It is **not** the runtime manifest for either app; install inside `backend/` and `frontend/`.

4. Getting Started

Prerequisites: Node 18+, a MongoDB URI, (optional) PostgreSQL 13+.

```
git clone <repo-url> && cd RootFin

# Backend
cd backend
npm install
cp .env .env.local    # then edit values (see §5)
npm run dev            # http://localhost:7000 (or PORT)

# Frontend (new terminal)
cd frontend
npm install
npm run dev            # http://localhost:3000
```

Then open `http://localhost:3000`. API docs: `http://localhost:<PORT>/api-docs`. Health: `GET /api/test`, DB state: `GET /api/db-status`.

Point the frontend at your local API by editing `frontend/src/api/api.js` (see §5.3).

5. Configuration

5.1 Backend environment variables (`backend/.env`)

`server.js` loads `.env.<NODE_ENV>` if that file exists, otherwise `.env`.

Variable	Used by	Description
<code>PORT</code>	<code>server.js</code>	listen port, default <code>7000</code>
<code>NODE_ENV</code>	everywhere	<code>development</code> (default) or <code>production</code>
<code>DB_TYPE</code>	<code>server.js</code>	<code>mongodb</code> (default), <code>postgresql</code> , or <code>both</code>
<code>MONGODB_URI</code>	<code>db/databases.js</code>	fallback Mongo connection string
<code>MONGODB_URI_PROD</code>	<code>db/databases.js</code>	used when <code>NODE_ENV=production</code>
<code>MONGODB_URI_DEV</code>	<code>db/databases.js</code>	used in non-production
<code>POSTGRES_DB_DEV</code> / <code>_USER_DEV</code> / <code>_PASSWORD_DEV</code> / <code>_HOST_DEV</code> / <code>_PORT_DEV</code>	<code>db/postgresql.js</code>	dev PG credentials (defaults: <code>rootfin_dev</code> , <code>postgres</code> , <code>postgres</code> , <code>localhost</code> , <code>5432</code>)
<code>POSTGRES_*_PROD</code> / <code>POSTGRES_URI_PROD</code> / <code>DATABASE_URL</code>	<code>db/postgresql.js</code>	production PG credentials or connection URI
<code>POSTGRES_URI_DEV</code>	<code>db/postgresql.js</code>	optional dev connection URI

Variable	Used by	Description
POSTGRES_LOGGING	db/postgre sql.js	true logs SQL
SYNC_DB	db/postgre sql.js	true runs <code>sequelize.sync()</code> on connect
e-mail settings	utils/emailS ervice.js	SMTP configuration for nodemailer

Values are supplied per environment and are never documented here.

Resilience features in `db/database.js` : environment-specific connection strings, protection against cross-environment connections, and `disconnected` / `reconnected` / `error` listeners.

Express middleware (`server.js`): JSON & urlencoded body limit **10 MB** (base64 attachments), `cookie-parser` , CORS with credentials.

CORS: an explicit origin allow-list (local dev and deployed frontends) is configured in `server.js` . Add new frontend origins there.

5.2 PostgreSQL pool / SSL

`db/postgresql.js` builds config per environment, supports a full URI with `dialectOptions` (SSL) and connection `pool` settings, and optionally syncs tables when `SYNC_DB=true` .

5.3 Frontend configuration

File	Purpose
<code>src/api/api.js</code>	exports <code>{ baseUrl }</code> , the API root used by every page. Set it to your local or deployed API (or switch to <code>import.meta.env.VITE_API_URL</code>).
<code>vite.config.js</code>	plugins <code>react()</code> and <code>tailwindcss()</code> ; dev server on port 3000 .
<code>vercel.json</code>	rewrites all paths to <code>/index.html</code> (SPA routing).
<code>eslint.config.js</code>	ESLint 9 flat config with react-hooks / react-refresh.
<code>src/config/salesInventoryAccess.json</code>	<code>allowedEmails</code> list — stores allowed to see Sales and Inventory menus (see §9). Case-insensitive; rebuild required in production.
<code>src/config/README.md</code>	how to add/remove stores from that list
<code>src/data/openingBalance.json</code>	opening balance seed data for stores

6. Backend in Detail

6.1 Startup flow (`server.js`)

1. Load env file → create Express app → `setupSwagger(app)` .
2. Apply body parsers, cookies, CORS, `OPTIONS *` preflight.

3. Mount routers (all under `/api` unless noted).
4. Utility endpoints: `GET /` (liveness text), `GET /api/test`, `GET /api/db-status` (reports Mongo `readyState` and PG authentication).
5. `app.listen` → connect DBs by `DB_TYPE`. PostgreSQL failure is non-fatal when Mongo is used; Mongo failure exits the process.

6.2 Routers → API surface

Router file	Base path(s)	Main endpoints
LoginRoute.js	/api (users)	POST /signin, POST /login, GET /getAllStores, GET /getAllUsers, PUT /updateUser/:id, DELETE /deleteUser/:id, POST /reset-password, POST /createPayment, GET /Getpayment, POST /saveCashBank, GET /getsaveCashBank, GET /pendingClosures, PUT /approveClosure/:id, GET /AdminColseView, GET /getTransactions, POST /syncTransaction, PUT /editTransaction/:id, GET /transaction/:id/attachment, GET /financialSummaryWithEdit
TwsRoutes.js	/api	GET /getEditedTransactions, GET /getsaveCashBank
DayBookRoutes.js	/api	GET /daybook, GET /daybook/range
ExpenseTargetRoutes.js	/api	POST/GET /expense-targets, GET /expense-targets/all
ShoeItemRoutes.js	/api	/shoe-sales/items, /:itemId, /:itemId/history
ItemGroupRoutes.js	/api	/shoe-sales/item-groups, /:id, /:id/items/:itemId/history, /:id/monthly-opening-stock
ManufacturerRoutes.js / BrandRoutes.js	/api	/shoe-sales/manufacturers[:id], /shoe-sales/brands[:id]
AddressRoutes.js	/api	/purchase/addresses[:id]
VendorRoutes.js	/api	/purchase/vendors[:id], /:vendorId/history
BillRoutes.js	/api	/purchase/bills[:id], /purchase/orders/:id/convert-to-bill, /purchase/receives/:id/convert-to-bill
VendorCreditRoutes.js	/api	/purchase/vendor-credits[:id], /next-number, /available, /apply-to-bill
PurchaseOrderRoutes.js	/api	/purchase/orders[:id], /number/:orderNumber, /next-number, POST /:id/send
PurchaseReceiveRoutes.js	/api	/purchase/receives[:id], /next-number, POST /:id/send

Router file	Base path(s)	Main endpoints
InventoryAdjustmentRoutes.js	/api	/inventory/adjustments[:id], /next-reference, /stock/item
TransferOrderRoutes.js	/api	/inventory/transfer-orders[:id], /:id/receive, /stock/item
StoreOrderRoutes.js	/api	/inventory/store-orders[:id], /next-number
StoreRoutes.js	/api	/stores[:id], /stores/loc/:locCode
SalesPersonRoutes.js	/api	/sales-persons[:id], /loc/:locCode
SalesInvoiceRoutes.js	/api	/sales/invoices[:id], POST /next-number
SalesReportRoutes.js	/api /reports/sales	/by-invoice, /summary, /by-item, /returns, /by-group
InventoryReportRoutes.js	/api /reports/inventory	/summary, /stock-summary, /valuation, /aging, /opening-stock, /stock-on-hand
ReorderAlertRoutes.js	/api	/reorder-alerts, /warehouse/:w, PUT /:id/notify, PUT /:id/resolve, DELETE /:id, POST /test-email
MergRoutes.js	/api	GET /mergedTransactions (currently commented out in server.js)

Other: /api-docs (Swagger UI), /api/test, /api/db-status.

6.3 Controllers (backend/controllers/)

Controller	Responsibility
LoginAndSignup.js	sign-up, login (bcrypt), user CRUD, password reset, store/user lists, payments
CloseController.js	cash/bank day closing, approval of pending closures, admin close view
DayBookController.js	Day Book per store/day and date ranges
TransactionController.js, TwsTransaction.js, TwsControllers.js, EditController.js	transaction sync, edit history, attachments, financial summary
ExpenseTargetController.js	monthly expense targets per store
ShoeItemController.js, ItemGroupController.js	items, groups, stock per warehouse, item history, monthly opening stock
ManufacturerController.js, BrandController.js	catalogue masters

Controller	Responsibility
VendorController.js , VendorHistoryController.js , AddressController.js	vendors, history log, addresses
PurchaseOrderController.js , PurchaseReceiveController.js , BillController.js , VendorCreditController.js	purchase cycle & numbering
InventoryAdjustmentController.js , TransferOrderController.js , StoreOrderController.js	stock movements
SalesInvoiceController.js , SalesPersonController.js	sales, returns, salespeople
SalesReportController.js , SalesByGroupController.js , InventoryReportController.js , OptimizedReportController.js	reports (cached)
StoreController.js	store master CRUD

6.4 Data layer

Mongoose models (backend/model/) User, Store, Transaction, TransactionHistory, Close (Closing.js), ExpenseTarget, Shoeltem, ItemGroup, ItemHistory, Manufacturer, Brand, Vendor, VendorHistory, VendorCredit, Address, PurchaseOrder, PurchaseReceive, Bill, InventoryAdjustment, TransferOrder, StoreOrder, SalesInvoice, SalesPerson, ReorderAlert, Counter (auto-number sequences).

Sequelize models (backend/models/sequelize/) — InventoryAdjustment, SalesInvoice, SalesPerson, Store, StoreOrder, Transaction, TransferOrder, User, Vendor, VendorCredit, VendorHistory, wired together in index.js .

6.5 Utilities (backend/utils/)

File	Role
stockManagement.js → improvedStockManagement.js → ultraEnhancedStockManagement.js	successive generations of the stock-update engine (per-warehouse quantity changes for sales, returns, receives, transfers, adjustments)
stockValidationSystem.js	validates stock consistency before/after operations
monthlyStockTracking.js	month-wise opening/closing stock
cashPropagation.js	carries closing cash/bank balances forward to later days
next*.js (SalesInvoice, PurchaseOrder, PurchaseReceive, CreditNote, StoreOrder, ItemGroup, InventoryAdjustment)	auto-increment document numbers via Counter
cacheManager.js	node-cache wrapper (TTL 300 s, generateCacheKey)
emailService.js , reorderNotification.js	nodemailer; low-stock reorder alerts
vendorHistoryLogger.js	vendor audit trail

6.6 Swagger

swagger.js builds a spec with swagger-jsdoc from JSDoc comments in route/*.js (e.g. in LoginRoute.js) and serves it at /api-docs .

7. Frontend in Detail

7.1 Bootstrapping

- `main.jsx` renders `<App />` in the router.
- `App.jsx` reads the logged-in user from client storage (`currentUser`), defines every `<Route>` , and wraps pages with guards.
- Unauthenticated users are redirected to `/login` ; authenticated users visiting `/login` are redirected to `/` .

7.2 Shared components (`src/components/`)

Component	Purpose
<code>Nav.jsx</code>	sidebar menu; applies role and <code>salesInventoryAccess</code> gating
<code>Header.jsx</code> , <code>Head.jsx</code>	top bar / document head
<code>DaybookGuard.jsx</code>	blocks pages until the previous day's Day Book is closed
<code>OptimizedTable.jsx</code>	paginated/virtualised data table
<code>AssignTargetModal.jsx</code>	set expense targets
<code>ImageUpload.jsx</code> , <code>SingleImageUpload.jsx</code> , <code>AttachmentDisplay.jsx</code>	base64 attachments
<code>LoadingScreen.jsx</code>	loading UI

7.3 Hooks (`src/hooks/`)

`useFetch` , `useOptimizedFetch` (cached fetching), `useFinancialData` , `useSidebar` , `useEnterToSave` , `usePreventNumberInputScroll` .

7.4 Utils (`src/utils/`)

`cache.js` (client cache), `customAlert.jsx` / `customConfirm.jsx` (styled dialogs), `imageUpload.js` , `warehouseMapping.js` (store ↔ warehouse names / location codes).

7.5 Page / route map (`src/pages/`)

Area	Routes	Pages
Auth	<code>/login</code>	Login
Home & Day Book	<code>/</code> , <code>/daybook</code> , <code>/datewisedaybook</code> , <code>/RentOutReport</code> , <code>/BookingReport</code>	Home, DayBook, Datewisedaybook, Booking
Accounts	<code>/income</code> , <code>/expenses</code> , <code>/Income&Expenses</code> , <code>/CashBankLedger</code> , <code>/securityReport</code> , <code>/Revenuereport</code>	Income, Expenses, SecurityReturn, SecurityPending, Security, Revenuereport, IncomeExpenseReport, BillWiseIncome

Area	Routes	Pages
Closing (admin)	<code>/CloseReport</code> , <code>/AdminClose</code> , <code>/PendingDaybookClosures</code>	CloseReport, AdminClose, PendingDaybookClosures
Store/User admin	<code>/ManageStores</code> , <code>/manage-users/*</code>	ManageStores, AddNewStore, AddNewUser, ExistingUsers, EditUser, ResetUserPassword
Items	<code>/shoe-sales/items</code> , <code>/item-groups</code> , <code>/price-lists</code> , <code>/inactive-items</code> , <code>.../stocks</code>	ShoeSalesItems, ShoeSalesItemCreate/Detail/DetailFromGroup, ShoeSalesItemGroups/Create/Detail, ShoeSalesPriceLists/Create, InactiveItems, ItemStockManagement, StandaloneItemStockManagement
Inventory	<code>/inventory/adjustments</code> , <code>/packages</code> , <code>/transfer-orders</code> , <code>/store-orders</code> , <code>/reorder-alerts</code>	InventoryAdjustments/Create/Detail, InventoryPackages/Create, TransferOrders/Create/View, StoreOrders/Create/View, ReorderAlerts
Sales	<code>/sales/customers</code> , <code>/orders</code> , <code>/invoices/returns</code>	/new
Purchase	<code>/purchase/orders</code> , <code>/receives</code> , <code>/bills</code> , <code>/payments</code> , <code>/vendor-credits</code> , <code>/vendors</code> (each with <code>/new</code> , <code>/:id</code> , <code>/:id/edit</code>)	PurchaseOrders/Create/Detail, PurchaseReceives/Create/Detail, Bills, BillDetail, BillBooking, PaymentsMade, VendorCredits, VendorCreditDetail, PurchaseVendors/Create/Detail
Reports	<code>/reports/sales</code> , <code>/sales-by-invoice</code> , <code>/sales-by-group</code> , <code>/inventory</code> , <code>/income-expense</code>	SalesReport, SalesByInvoiceReport, SalesByGroupReport, InventoryReport, IncomeExpenseReport, OptimizedFinancialSummary

7.6 PDF / print

Day Book and report PDFs use `html2canvas-pro` + `jspdf` (fits to a single page; supports Tailwind 4 oklch colours). Print views use `react-to-print`.

7.7 Tests

`src/pages/__tests__` contains tests for transfer-order creation and warehouse filtering. No test runner script is defined in `package.json`.

8. Domain Modules

1. **Day Book & Closure** – each store records transactions (income, expense, payments, security deposits/returns, bookings). At day end the store submits a *cash/bank closing*; supervisors/admins approve in *Pending Daybook Closures*. `cashPropagation` carries balances forward. `DaybookGuard` forces yesterday to be closed first.
2. **Expense Targets** – per-store monthly targets (`AssignTargetModal`).
3. **Items & Groups** – items belong to item groups with per-warehouse stock, price lists, brands, manufacturers, history logs and monthly opening stock.
4. **Purchase cycle** – Vendor → Purchase Order → Purchase Receive (adds stock) → Bill → Payments Made; Vendor Credits applied to bills; vendor history log.
5. **Inventory control** – adjustments, inter-warehouse transfer orders (with receive step), store orders, reorder alerts with e-mail.

6. **Sales** – invoices (auto-numbered), returns, credit notes, salespeople per location, sales reports by invoice / item / group.
7. **Reporting** – inventory valuation, aging, stock-on-hand, opening stock; sales summaries; financial summary; results cached for 5 minutes.

9. Roles, Access Control & Guards

- `power: 'admin'` – full access: user/store management, close reports, stock-management pages, Purchase section.
- Head-office location users – may access `AdminClose`.
- `role: 'supervisor'` (and admin) – `PendingDaybookClosures`.
- **Cluster manager** (`isClusterManager`) – redirected to `/datewisedaybook` home; `ClusterGuard` limits income/expense pages.
- **Sales & Inventory visibility** – menu items appear only for emails in `frontend/src/config/salesInventoryAccess.json`.
- **DaybookGuard** – wraps most operational pages (requires previous-day close).

10. CI/CD & Deployment

Piece	Details
Frontend	Vercel; <code>vercel.json</code> rewrites to <code>index.html</code> .
Backend	Render and/or AWS EC2.
<code>.github/workflows/deploy.yml</code>	On push to <code>master</code> : checkout → configure SSH from repository secrets → <code>rsync</code> to the server → <code>npm install</code> → restart via PM2. Credentials are stored as GitHub Actions secrets.
<code>.github/workflows/daily-commit-report.yml</code>	scheduled; lists recent commits and flags those not matching Conventional Commits (<code>type(scope): summary</code>) in the run summary.

Production start: `NODE_ENV=production node --max-old-space-size=2048 server.js`.

11. Maintenance Scripts (`backend/scripts/`, run with `node`)

Script	Purpose
<code>check-database-connection.js</code>	verify DB connectivity
<code>create-indexes.js</code> , <code>quick-optimize.js</code> , <code>optimize-data-types.js</code>	index / performance tuning
<code>sync-stores-to-postgresql.js</code> , <code>migrate-vendors-to-postgresql.js</code> , <code>sync-vendor-history-table.js</code>	Mongo → PostgreSQL migration
<code>add-vendor-credit-columns.js</code> , <code>drop-invoiceno-unique-index.js</code>	schema fixes
<code>analyze-cash-closecash-issue.js</code> , <code>fix-cash-closecash-swap.js</code> , <code>MIGRATION_CASH_CLOSECASH_README.md</code>	repair swapped cash/closeCash values

Script	Purpose
<code>recalculate-close-balances.js</code>	rebuild closing balances
<code>fix-return-location-codes.js</code> , <code>test-return-query.js</code> , <code>check-transfer-orders.js</code> , <code>test-calc.js</code>	data fixes / diagnostics

Also in `backend/`: `query_closures.js`, `query_targets.js`, `check_palakkad.js`, `revoke_requests.cjs`, `test_db.js` (ad-hoc diagnostics).

12. Housekeeping Files & Known Issues

- **Dev helpers (safe to ignore / delete):** `fix_home.cjs`, `fix_report.cjs`, `scratch.jsx`, `production_changes.patch`, `frontend/fix_*.cjs`, `frontend/fix_*.py`, `frontend/patch_css.js`, `frontend/test-*.js`, `frontend/src/pages/fix*.cjs`, `pages/InactiveItems*.txt`.
 - `frontend/src/pages/S极狐alesInvoiceCreate.jsx` has a corrupted filename (stray characters) — it is a stray duplicate of `SalesInvoiceCreate.jsx`.
 - `backend/utlis/` (typo) and `backend/utlis/` coexist.
 - `frontend/Readme.txt` and old README content describe an earlier plan (JWT, Plaid/Stripe) that is **not** implemented; this document supersedes it.
 - The API URL is set directly in `api.js`; consider `VITE_API_URL`.
 - `server.js` contains a large commented-out legacy version at the bottom.
 - Backend uses Express 4; root `package.json` lists Express 5 (unused).
-

13. Git Workflow

- Default deploy branch: `master`; feature branch carries local features merged with production fixes (see `production_changes.patch`).
- Use Conventional Commits (`feat:`, `fix:`, `docs:` ...) — audited daily by the commit-report workflow.

Contact (legacy README): Rootments — Project dev: Jishnu M. License: MIT — Rootments.